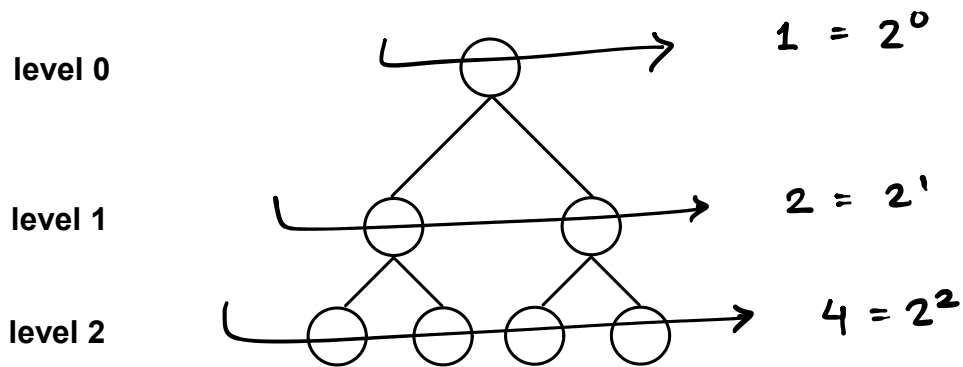


Perfect Binary Tree

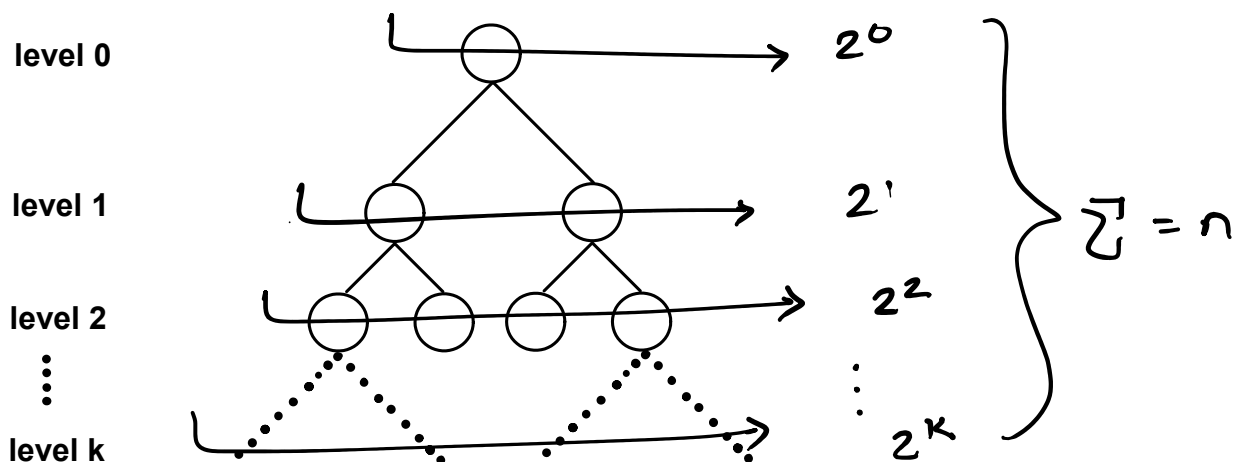
$$i \longrightarrow 2^i$$

Perfect Binary Trees

A binary tree is said to be a **perfect binary tree** if **all its levels** are completely **filled** i.e. every internal node in the binary tree has **exactly two child nodes**.



The **height** of a perfect binary tree is **logarithmic**.



$$2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^k = n$$

This is a **finite GP** where,

$$\begin{aligned} a &= 2^0 = 1 \\ r &= 2 \\ \# &= k+1 \end{aligned}$$

The sum of finite GP given as follows

$$= a * [(r^{\#}-1) / (r-1)]$$

highest level of
a tree

Substituting the values of a, r, and #, we get
 $= 1 * [(2^{k+1}-1) / (2-1)]$
 $= 2^{k+1}-1$
 $\sim 2^k$

=
 Height of Tree

Therefore,

$$\begin{aligned} 2^k &= n \\ k &= \log_2 n \\ \Rightarrow & \boxed{} \end{aligned}$$

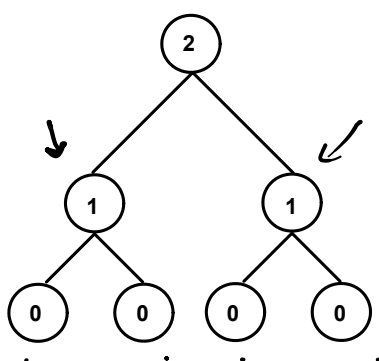
The number of nodes in a perfect binary tree that has n nodes at a height h is

$$\left\lceil \frac{n}{2^{h+1}} \right\rceil \Leftarrow$$

$$\begin{aligned} n &= 7 \\ h &= 0 & \left\lceil \frac{7}{2^{0+1}} \right\rceil \end{aligned}$$

$$h = 1 \quad \left\lceil \frac{7}{2^{1+1}} \right\rceil$$

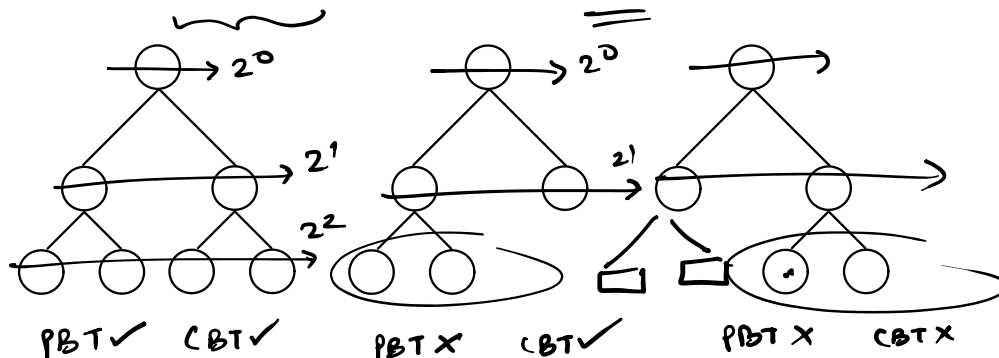
$$h = 2 \quad \left\lceil \frac{7}{2^{2+1}} \right\rceil$$



height	#
0	4 ✓
1	2 ✓
2	1 ✓

Complete Binary Trees

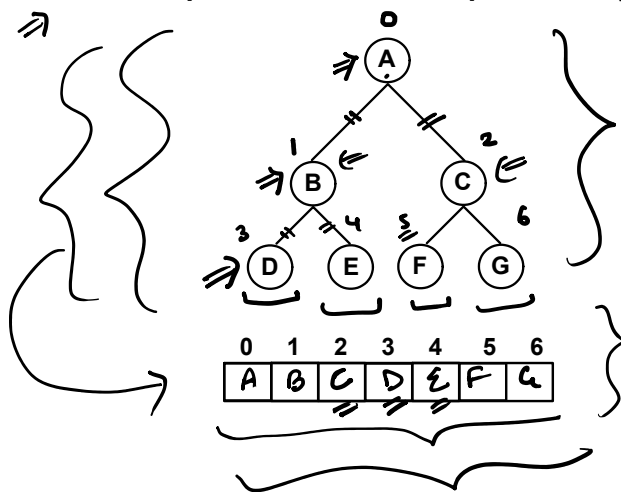
A binary tree is said to be a **complete binary tree** if **all its levels** are completely **filled** except for the **last level** which **may or may not** be filled. And, in case the last level is **partially** filled then it is filled in the **left to right** direction.



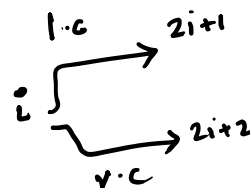
→ The **height** of a complete binary tree is **logarithmic**.

→ The max. no. of nodes in a complete binary tree that has **n** nodes at a height **h** is $2^{h+1} - 1$

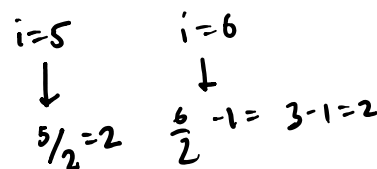
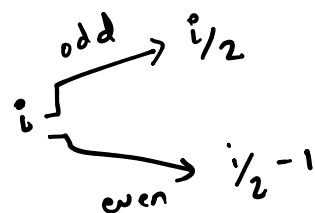
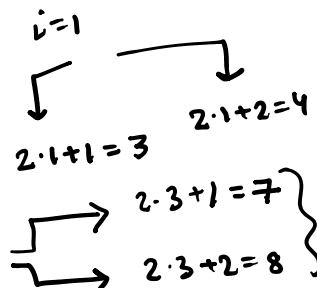
Linear Representation of Complete Binary Trees



Logical View



Internal View



Heap

It is a kind of **complete binary tree** in which every node satisfies the **heap property**.

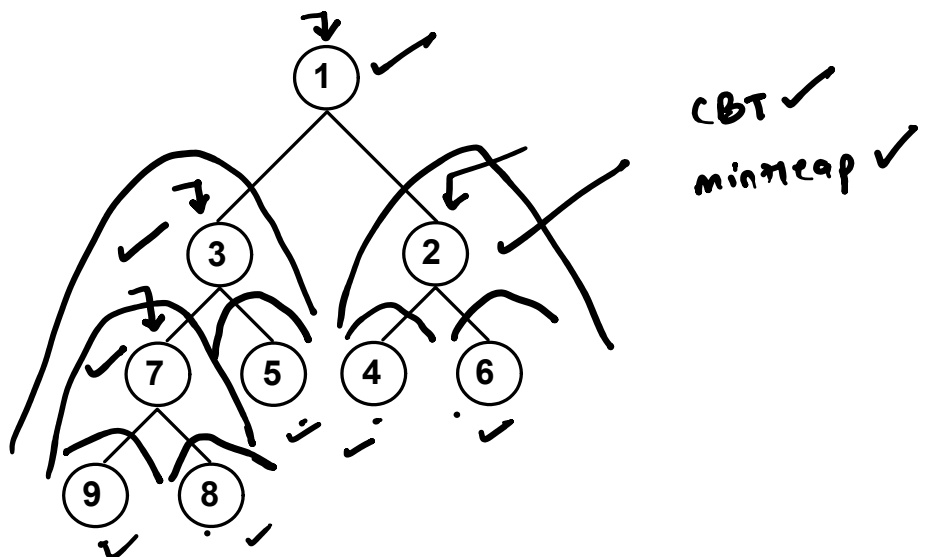
Types of Heap

- ⇒ **Min-Heap** - optimized for operations on **minimum** element
- ⇒ **Max-Heap** - optimized for operations on **maximum** element

Min-Heap

It is a kind of **complete binary tree** in which every node satisfies the **min-heap property**.

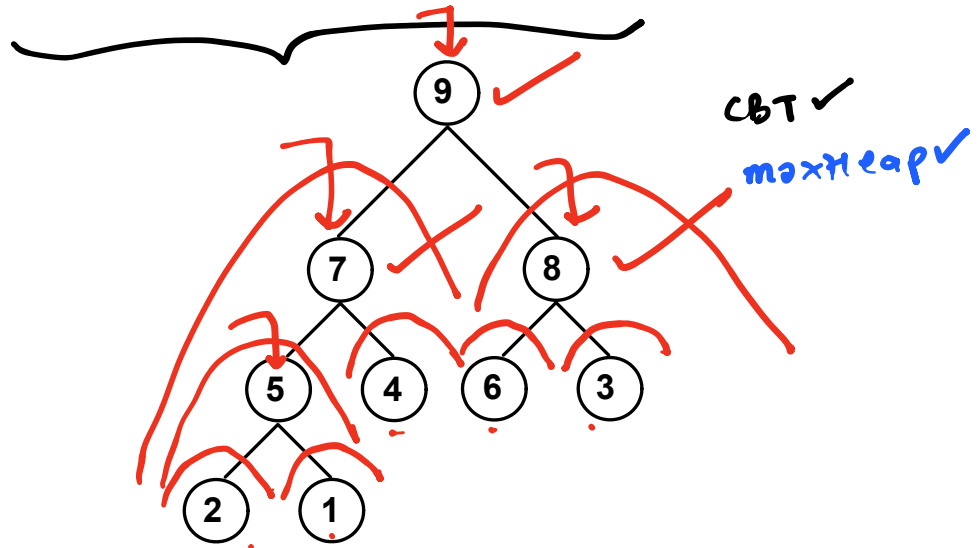
The **min-heap property** states that **value of each node** in the heap must be **less than** the value of all the nodes in its **left subtree** and its **right subtree**.



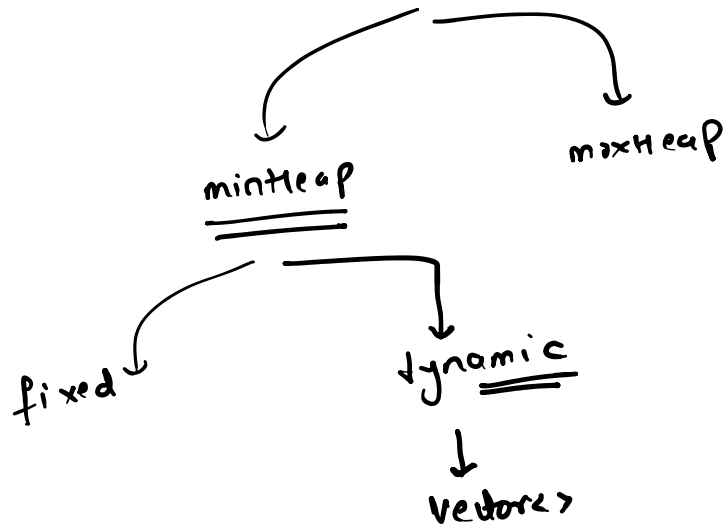
Max-Heap

It is a kind of **complete binary tree** in which every node satisfies the **max-heap property**.

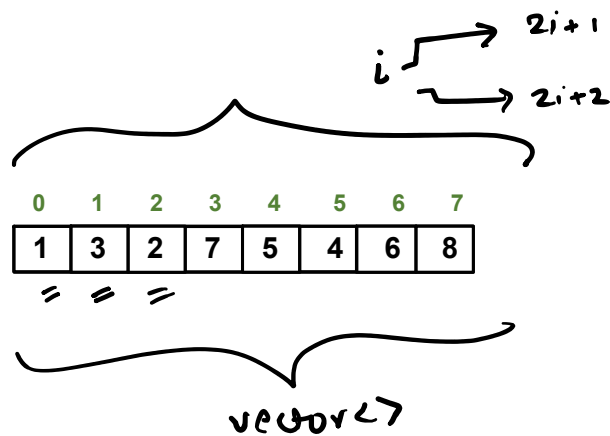
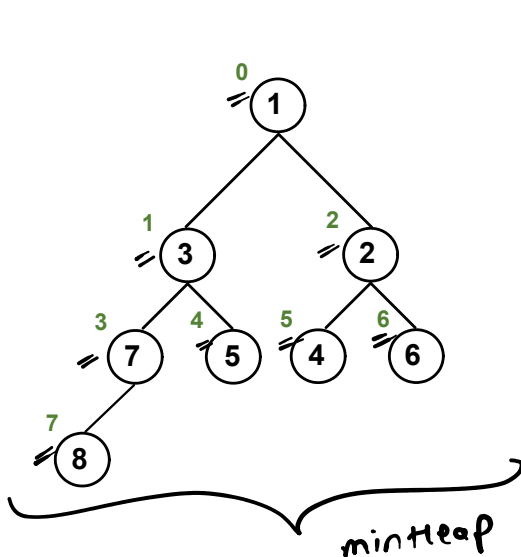
The **max-heap property** states that **value of each node** in the heap must be **greater than** the value of all the nodes in its **left subtree** and its **right subtree**.



Implementation of Heaps



Linear Representation of Heaps



$$\begin{aligned}
 &\text{odd } i \rightarrow i/2 \\
 &\text{even } i \rightarrow i/2 - 1
 \end{aligned}$$

$$2i+1, 2i+2 < n$$

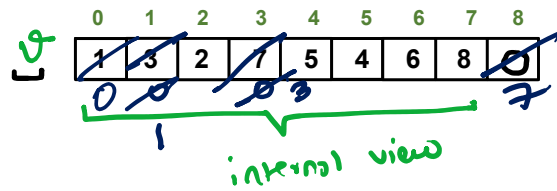
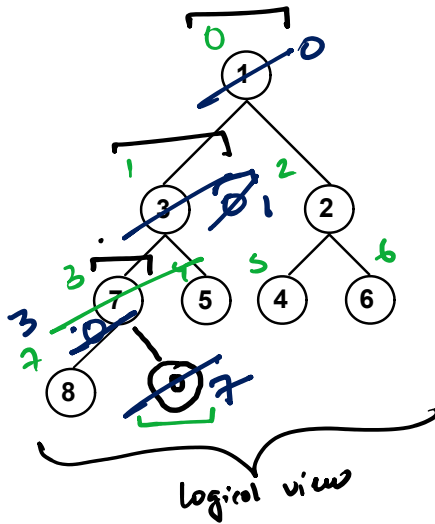
Heap Operations

1. **Push** : to insert an element onto the heap.
2. **Pop** : to delete and return the smallest/largest element from the heap.
3. **Top** : to access the smallest/largest element without popping from the heap. $\log n$
4. **Size** : return the size of the heap $\log n$
5. **Empty** : return **True** if heap is empty, **False** otherwise. const

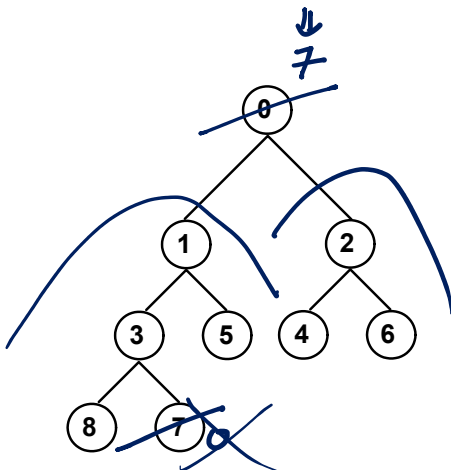
const

const

Heap Push Operation



Heap Pop Operation

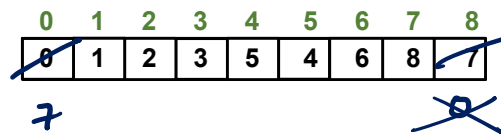


1. $\text{swap}(v[0], v[n-1])$

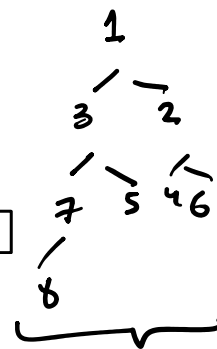
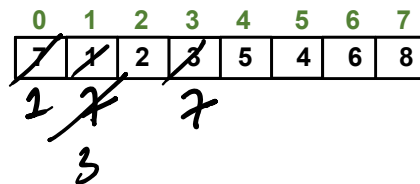
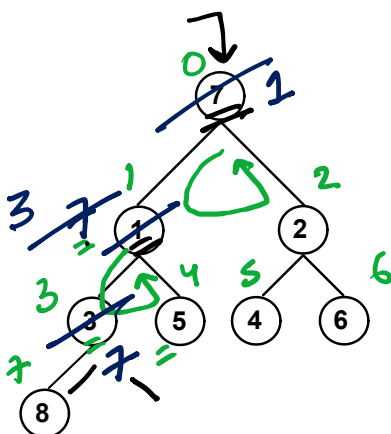
2. $v.\text{pop_back}()$

3. fix the min-heap prop. at the root

} $\text{heapify}(0)$



$i = 0$



Merge Ropes

Given the lengths of N ropes, design an algorithm to compute the **minimum cost** to connect these N ropes into one rope such that any point of time you can only merge **two ropes** and cost to connect two ropes is equal to the **sum of their lengths**.

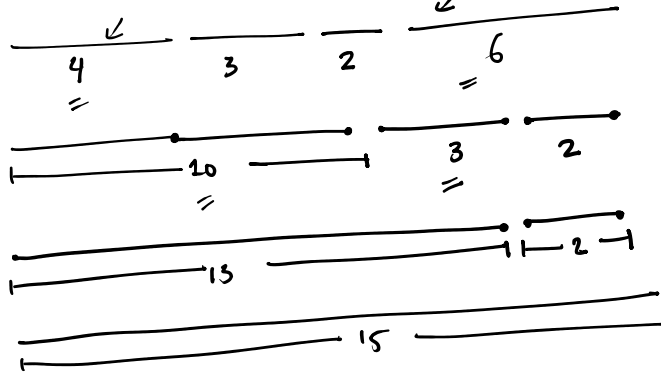
Example

Input : [4, 3, 2, 6]
Output : 29

$$\begin{aligned} \Rightarrow 2 + 3 &= 5 \\ \Rightarrow 4 + 5 &= 9 \\ \Rightarrow 9 + 6 &= 15 \\ &\underline{29} \end{aligned}$$

$$\begin{aligned} 2 + 3 &= 5 \\ 4 + (2 + 3) &= 9 \\ (4 + (2 + 3)) + 6 &= 15 \end{aligned}$$

$\eta = 4$
4, 3, 2, 6
4, 5, 6
9, 6
15



[2, 3, 4, 6]
[4, 5, 6]
[6, 9]

$\Rightarrow$ 1. sort : nlogn
repeat the

following steps $n-1$ times

1. pick $arr[0], arr[1]$ $\leftarrow$ const
2. $merge_cost = arr[0] + arr[1]$ $\leftarrow$ const
3. $total += merge_cost$ $\leftarrow$ linear
4. insert new rope in sorted set $\leftarrow$ const

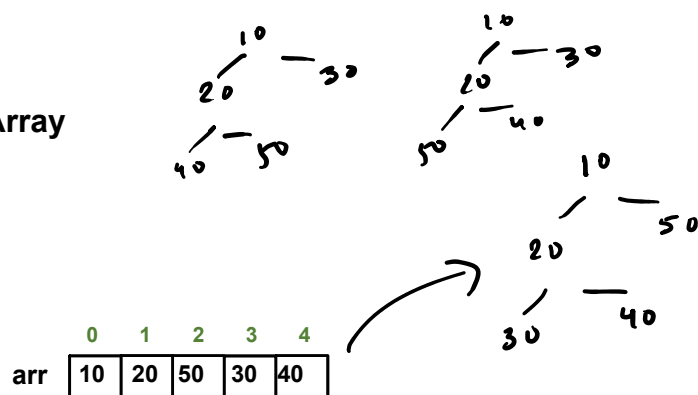
it remains sorted

$$10 + 3 = 13$$

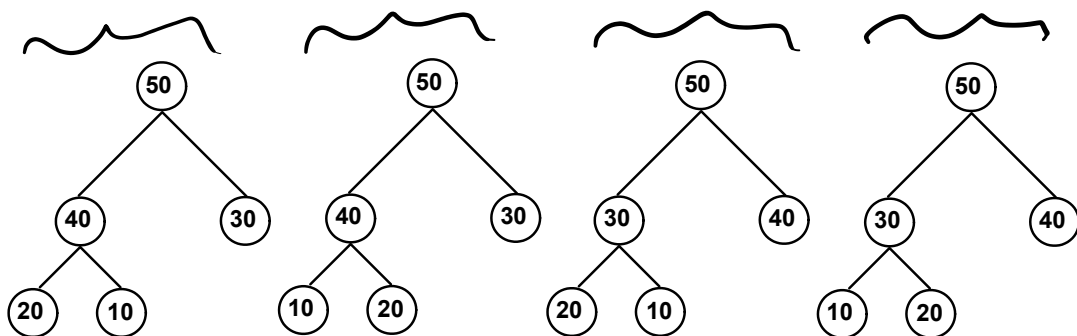
$$13 + 2 = 15$$

$$\underline{38}$$

Building Heap from an Array

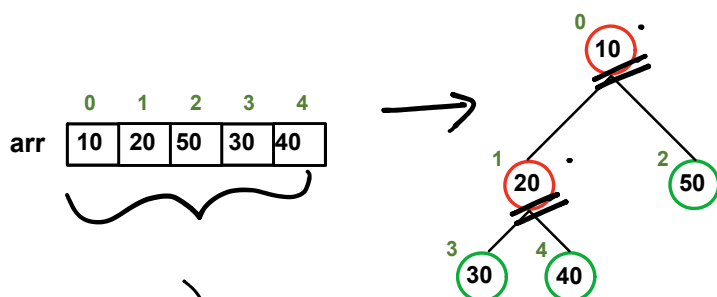


$\{10, 20, 50, 30, 40\}$



arr

0	1	2	3	4
10	20	50	30	40

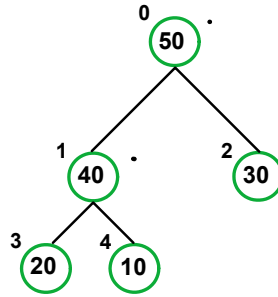


time: $O(n \log n)$

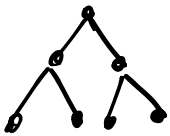
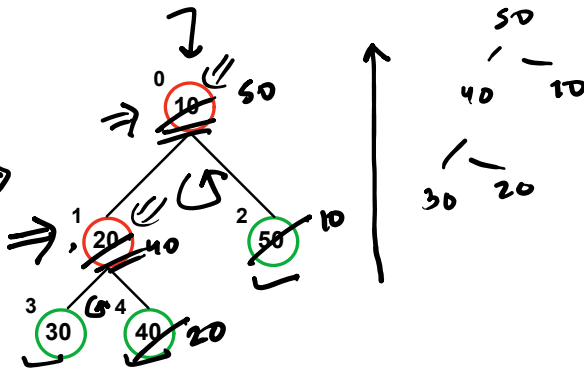
Sort
in $\downarrow$ ing
order

time: $O(n \log n)$

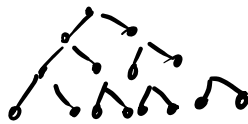
	0	1	2	3	4
arr	50	40	30	20	10



	0	1	2	3	4
arr	10	20	50	30	40



$n = 7$
 $\text{leaf} = 4$
 $\text{non-leaf} = 3$



$n = 15$
 $\text{leaf} = 8$
 $\text{non-leaf} = 7$

$\text{non-leaf} = n/2$
 $\text{leaf} = n - n/2$

$n/2 * \text{time for heapify}$

~~$n/2 * \log n$~~

~~$n \log n$~~

The **heapify** operation, in **worst case**, takes time proportional to the **height** of the **node** on which it **invoked**.

The maximum number of nodes in a **complete binary tree** at a height $h \sim n / 2^{h+1}$

The total amount of work done at nodes which are at **height** h , =

$$(n / 2^{h+1}) * O(h)$$

Therefore, the total amount of work done at all the non-leaf nodes,

$h = 1 \text{ to } \log n$

$$\sum_{h=1}^{\log n} (n / 2^{h+1}) * O(h)$$

$$\begin{aligned} & \Rightarrow \sum_{h=1}^{\log_2 n} (n / 2^{h+1}) * O(h) \quad (=) \\ & < \sum_{h=0}^{\log_2 n} (n / 2^{h+1}) * O(h) \quad (\swarrow) \\ & \Rightarrow < \sum_{h=0}^{\infty} (n / 2^{h+1}) * O(h) \quad (\searrow) (=) \\ & = \sum_{h=0}^{\infty} (n / (2 \cdot 2^h)) * h.c \\ & = n c / 2 \sum_{h=0}^{\infty} (1 / 2^h) * h \\ & = n c / 2 \sum_{h=0}^{\infty} (h / 2^h) \\ & = n.c \sim O(n) \end{aligned}$$

$$\sum_{h=0}^{\infty} \binom{n}{h} \cdot h c$$

$$h=0 \quad 2^n \cdot 2$$

$$\frac{nc}{2} \sum_{h=0}^{\infty} \frac{h}{2^n}$$

$$\frac{nc}{2} \cdot 2 \sim c$$

$$\underbrace{\frac{0}{2^0} + \frac{1}{2^1} + \frac{2}{2^2} + \dots}_{\sim 2}$$

Heap Sort Algorithm

Given an array of n integers, sort them using the heap-sort algorithm.

Example :

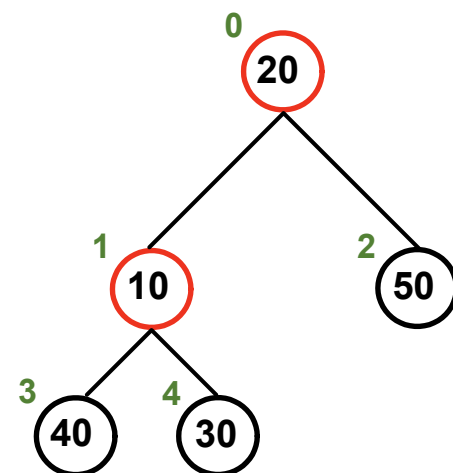
Input :

0	1	2	3	4
20	10	50	40	30

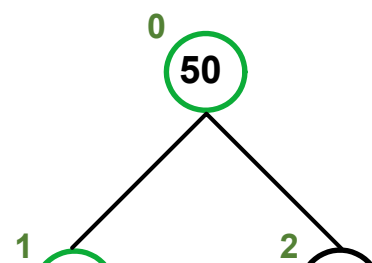
Output :

0	1	2	3	4
10	20	30	40	50

0	1	2	3	4
20	10	50	40	30



0	1	2	3	4
50	40	20	10	30



50	40	20	10	30
----	----	----	----	----

